

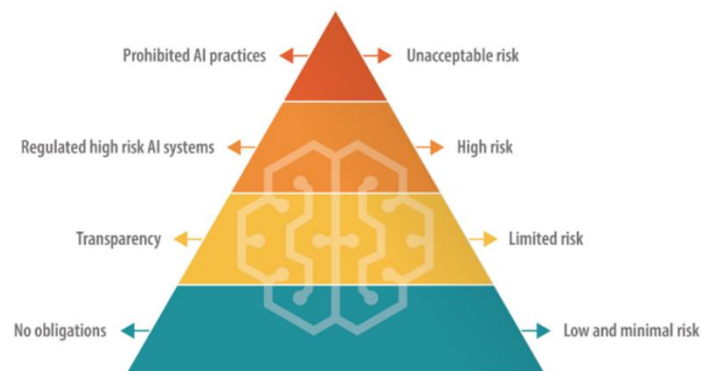
MACHINE LEARNING FOR BIOENGINEERING - Mod 1

Introduction to Machine Learning:

AI has boomed in the 2010s thanks to an increase in computational power, mainly focused on the growth of GPU and TPU capabilities and cloud infrastructures, an explosion in big data, which represent the “money” of this field of tech, shambles in machine learning algorithms, arrival of open-source ecosystems and new investments and industry adoption.

AI, according to a European act, cannot be used in every field and, also where it can be used, some policies and regulations are needed:

- Applications with low and minimal risk, such as AI-enabled video games or spam filters, have no obligations.
- Applications with a limited risk, such as chatbots, are necessary to ensure transparency, so users should be aware that they are interacting with a machine, in order to allow them to take an informed decision to continue or step back.
- Applications with high risk, like safety components of products or critical infrastructures that, in case of malfunctioning, could put the life and health of citizens at risk, need a specific regulation.
- Applications with unacceptable risk, so the ones with a clear threat to the safety, livelihoods and rights of people, are prohibited for AI.



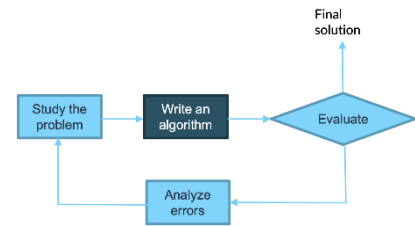
What is AI? AI is the theory and development of computer systems able to perform tasks normally requiring human intelligence.

Machine learning, instead, is a subset of AI techniques which use statistical learning to enable backend to improve with experience: in this sense, the more data that we have, the better the performance.

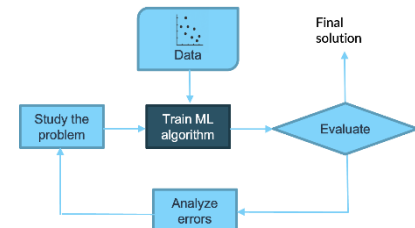
Finally, deep learning is a subset of ML techniques which use complex algorithms, such as neural networks, to learn from large amounts of data.

What is Machine Learning? Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed. (Cit. Arthur Samuel)

The standard approach for carrying out a task is to study the problem we want to solve, write an algorithm and then evaluate the result to analyze errors and, eventually, define better the problem and, consequently, change the algorithm until finding the final solution.



The approach of machine learning, instead, starts with the study of the problem, followed by training of the ML algorithm thanks to the experimental data with a subsequence evaluation and analysis of errors that allows us to eventually correct the study of the problem and the algorithm.



In this sense, data are our main source for our algorithms to work correctly and, in particular, data preparation is a highly time-invasive part of any machine learning project: in fact, it is at first necessary to extract feature, which is a task that requires human knowledge.

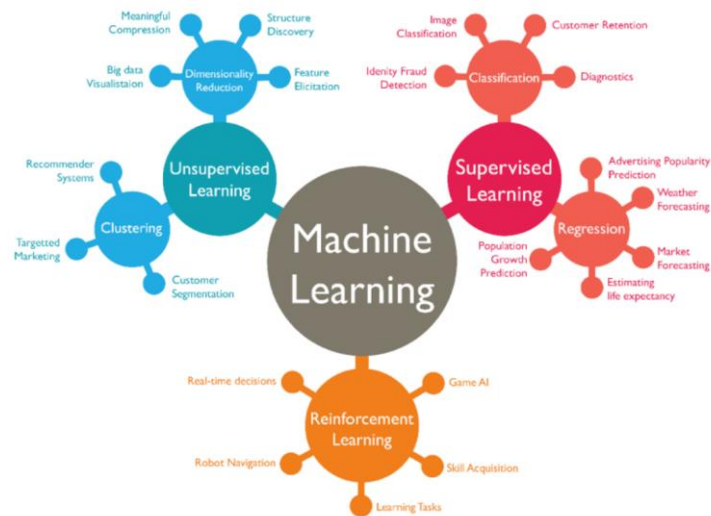
Feature extraction is carried out by engineers after discussing with doctors to understand which features are the best according to the information we want to obtain.

Feature extraction is crucial: information must be useful and it depends on the problem we want to investigate, so problem definition is extremely important.

Deep learning algorithms can solve the problem of features extraction: with deep learning, in fact, the system is able to learn how to extract information by raw data and, with them, to take decision regarding the initial problem.



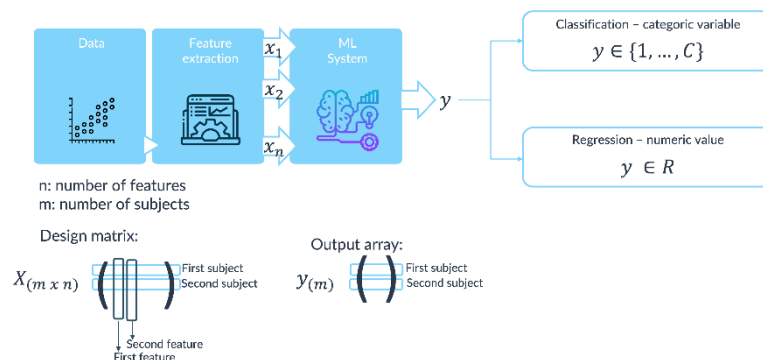
Supervised Learning (definition and model evaluation):



Machine Learning algorithms can be divided into three main categories:

1. Unsupervised learning, in which we do not have the solutions during the training.
2. Supervised learning, in which we have also the solutions during the training.
3. Reinforcement learning, based on rewards and penalties given according to each step.

So, in supervised learning the training data that we feed to the algorithm include the states output, called labels and the typical task for which we exploit this model of training is predicting future observations: in case the label is a categorical variable, we talk about classification, while if the label is a numeric value, we talk about regression.



Another definition for machine learning is: “A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E”. (Cit. Tom Mitchell)

From this definition we understand that, when designing an ML algorithm, we have to define:

1. Task T: we must understand precisely which is the input and which is the output wanted.
2. Experience E: how much data do we have? Are they enough? Are they good?
3. Performance metric P: we need to introduce some metrics for measuring quantitatively the performance.

If our task is a regression task, we have different metrics that we can use to assess performance:

• **MAE** (Mean Absolute Error):

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i|$$

• **MSE** (Mean Squared Error):

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

• **RMSE** (Root Mean Squared Error):

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2}$$

Regression	
'explained_variance'	metrics.explained_variance_score
'max_error'	metrics.max_error
'neg_mean_absolute_error'	metrics.mean_absolute_error
'neg_mean_squared_error'	metrics.mean_squared_error
'neg_root_mean_squared_error'	metrics.mean_squared_error
'neg_mean_squared_log_error'	metrics.mean_squared_log_error
'neg_median_absolute_error'	metrics.median_absolute_error
'r2'	metrics.r2_score
'neg_mean_poisson_deviance'	metrics.mean_poisson_deviance
'neg_mean_gamma_deviance'	metrics.mean_gamma_deviance
'neg_mean_absolute_percentage_error'	metrics.mean_absolute_percentage_error
'd2_absolute_error_score'	metrics.d2_absolute_error_score
'd2_pinball_score'	metrics.d2_pinball_score
'd2_tweedie_score'	metrics.d2_tweedie_score

On the other hand, if the task is a classification task, we need to define other metrics: in particular, we exploit the Receiver Operating Characteristic (ROC) analysis.

ROC analysis is a standard method for evaluating and comparing the performance of classification models, and its main limitations rely solely on accuracy, especially in imbalanced datasets. Originally developed in signal detection theory to illustrate the trade-off between true positive rate (hit rate) and false positive rate (false alarm).

Consider the case where our classifier is dichotomous: each instance, term used to refer to a general input-output pair, is mapped to an element of the set $\{p, n\}$, representing the positive and negative classes, respectively.

(An instance is a vector of features describing a subject and is the input to which the classifier, so the ML algorithm, has to assign an output that must be a class: some classifiers output a continuous score, so in this case is required to apply a threshold to decide class membership of the output found, while other classifiers produce a discrete class label directly.)

Let us denote the predicted classes with the set $\{Y, N\}$:

- Y means the classifier predicts the subject has a disease.
- N means the classifier predicts the subject is healthy.

Given a dichotomous classifier and a single instance, there are four possible outcomes:

- The instance is positive and classified as positive: True Positive (TP).
- The instance is positive and classified as negative: False Negative (FN).
- The instance is negative and classified as negative: True Negative (TN).
- The instance is negative and classified as positive: False Positive (FP).

Given a classifier and a set of instances (test set), a confusion matrix (2x2) can be constructed and each of its cells represents a conditional probability, which express the probability that the class label predicted by the classifier is correct, so it is equal to the one actually assigned to the instance.

Actual \ Predicted	Positive (Y)	Negative (N)
Positive (p)	$P(Y p)$ TP rate	$P(N p)$ FN rate
Negative (n)	$P(Y n)$ FP rate	$P(N n)$ TN rate

- TP rate = Positives correctly classified / Total positives.
- FP rate = Negatives incorrectly classified / Total negatives.
- TN rate = Negatives correctly classified / Total negatives.
- FN rate = Positives incorrectly classified / Total positives.

If the classifier is perfect, we obtain only correct predictions, obtaining a confusion matrix with:

$$P(Y|p) = 1, \quad P(Y|n) = 0, \quad P(N|n) = 1, \quad P(N|p) = 0$$

If the classifier isn't perfect, these probabilities take values strictly between 0 and 1. The probabilities are related by the following identities:

$$P(Y|p) + P(N|p) = 1 \quad \text{and} \quad P(Y|n) + P(N|n) = 1$$

Given the confusion matrix we can estimate some useful parameters:

- **Sensitivity (or Recall or True Positive Rate):** proportion of actual positives correctly identified

$$TPR = P(Y|p) = \frac{TP}{TP + FN}$$

- **Specificity (or True Negative Rate):** proportion of actual negatives correctly identified

$$TNR = P(N|n) = \frac{TN}{TN + FP}$$

- **Precision (or Positive Predictive Value):** proportion of predicted positives that are actually positive

$$\text{Precision} = PPV = P(p|Y) = \frac{TP}{TP + FP}$$

We can also estimate accuracy, which measures the overall proportion of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

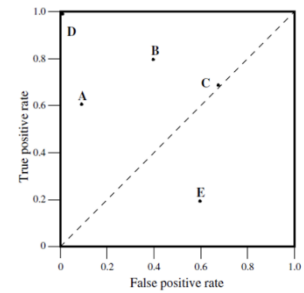
But imbalanced datasets can be misleading and, so, in these cases we use the F1-score, which balances precision and recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1-score is defined as the harmonic mean of Precision and Recall and it penalizes large differences between Precision and Recall, highlighting poor performance when either metric is low.

The confusion matrix has only two degrees of freedom, in fact it has only two independent values: this means that all the information it contains can be mapped to a point in a 2D graph.

In particular, if we plot on the x-axis the False Positive Rate and on the y-axis the True Positive Rate, we can obtain the ROC plane: the perfect classifier is the one that is located higher and to the left in this plane, while classifiers down and to the left are more conservative, so they make positive predictions only when there is strong evidence, and classifiers up and to the right are more liberal, so they tend to predict the positive class even with weak evidence.



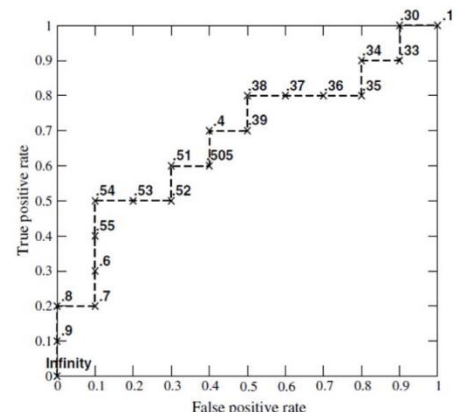
In many real-world problems with a majority of negative instances, performance in the left region of the ROC space is often more important.

A classifier on the main diagonal of the ROC plane is a random classifier and makes predictions without using any informative pattern from the data, so it has no discrimination power: any classifier that lies below this diagonal performs worse than random.

What we've said so far about the ROC curve can be applied only to classifiers that produce a single class label for each instance, not a score or a probability, since they yield a single point, not a full curve as, instead, do the classifiers that produce a continuous output.

In fact, the output produced by the last category of classifiers can be converted into a class, so into a binary decision, by using a threshold: for each threshold value we'll obtain a different point in the ROC plane and, by varying the threshold from minus infinite to plus infinite, we can generate a series of points and trace a curve in the ROC plane, which is the ROC curve.

Any ROC curve generated by a finite number of instances is a step curve: as the number of instances tends to infinite, the ROC curve approaches a smooth, continuous curve.



Setting a threshold equal to infinite corresponds to predicting all instances as negative, while as the threshold decreases, more instances are predicted positive: curves move up and to the right, and this transition reflects the move from a conservative to a liberal classification behavior.

If we want to obtain a ROC curve also for the dichotomous classifier that gives as output a binary class label directly, we must convert this categorical output into a score.

Fortunately, many dichotomous classifiers can be converted into scoring classifiers by "looking inside": in particular, we can look at proportions of positive samples, fraction of neighbors or confidence values. Going deeper in details regarding the classifiers that produce as output not a single class label directly but, instead, a score or a probability, it is important to say something about the last ones, so the classifiers that estimate the probability of class membership and then apply a decision rule in order to assign a specific class to that instance.

We have, in fact, to say that the probability estimates may not be perfectly calibrated: calibration means that the predicted probabilities match the observed frequencies.

For these types of classifiers, also, it is not optimal to use a fixed threshold but, instead, it is better to explore alternative thresholds and analyze the trade-offs between true positives and false positives.

ROC curves have a very desirable characteristic: they are insensitive to changes in class, which is the relationship between the rows of the confusion matrix, so actual positives and actual negatives. Instead, some metrics that use values from both rows, such as accuracy, are affected by class imbalance: as the class distribution changes, those metrics will also change, even if the classifier's behavior stays the same.

We can conclude that the ROC curve is a 2D plot that summarizes the performance of a classifier across all thresholds: to compare classifiers with a single number, we compute the Area Under the ROC Curve (AUC), whose statistical interpretation says that AUC is the probability that the classifier assigns a higher score to a randomly chosen positive instance than to a randomly chosen negative one. AUC ranges from 0 to 1: if it is 1, it means we have a perfect classifier, while if it's equal to 0.5, it means that we have a random classifier, so every value between 0.5 and 1 represents a classifier with a predictive value.

In this sense, the ideal ROC curve connects the points (0,0), (0,1) and (1,1), enclosing the maximum possible area (so $AUC = 1$); if two ROC curve don't intersect, the one that lies consistently above the other represents the better classifier, while, if the ROC curves intersect, the comparison depends on the application's priorities.

Scoring	Function	Comment
Classification		
'accuracy'	<code>metrics.accuracy_score</code>	
'balanced_accuracy'	<code>metrics.balanced_accuracy_score</code>	
'top_k_accuracy'	<code>metrics.top_k_accuracy_score</code>	
'average_precision'	<code>metrics.average_precision_score</code>	
'neg_brier_score'	<code>metrics.brier_score_loss</code>	
'f1'	<code>metrics.f1_score</code>	for binary targets
'f1_micro'	<code>metrics.f1_score</code>	micro-averaged
'f1_macro'	<code>metrics.f1_score</code>	macro-averaged
'f1_weighted'	<code>metrics.f1_score</code>	weighted average
'f1_samples'	<code>metrics.f1_score</code>	by multilabel sample
'neg_log_loss'	<code>metrics.log_loss</code>	requires predict_proba support
'precision' etc.	<code>metrics.precision_score</code>	suffixes apply as with 'f1'
'recall' etc.	<code>metrics.recall_score</code>	suffixes apply as with 'f1'
'jaccard' etc.	<code>metrics.jaccard_score</code>	suffixes apply as with 'f1'
'roc_auc'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovr'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovo'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovr_weighted'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovo_weighted'	<code>metrics.roc_auc_score</code>	

Overview of classification scoring options in `scikit-learn`'s model selection tools

Supervised Learning (generalization):

Linear regression without bias:

$$\hat{y} = w_1 x_1 + w_2 x_2$$

Vector notation:

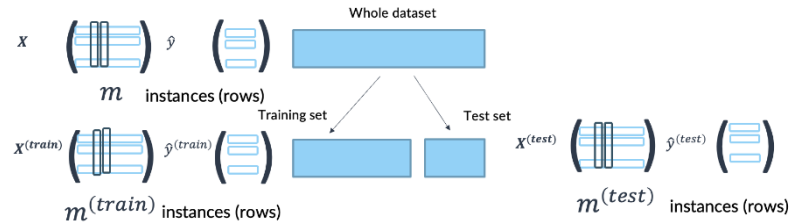
$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \in \mathbb{R}^2$$

$$\hat{y} = \mathbf{w}^T \mathbf{x}$$

General case with n features:

$$\mathbf{x} \in \mathbb{R}^n, \quad \mathbf{w} \in \mathbb{R}^n, \quad \hat{y} = \mathbf{w}^T \mathbf{x}$$

Both $\mathbf{x}$ and $\mathbf{w}$ are column vectors.
 $\mathbf{w}$ is the vector of parameters (i.e., the set of weights)



The whole dataset is divided into two, forming the training set and the test set; after this division, the ML algorithm, which consists in an algorithm for performing predictions through linear regression, is trained using the training set and the objective of this phase is to minimize the Mean Squared Error.

Training objective: minimize the Mean Squared Error on the training set

$$\text{MSE}_{\text{train}} = \frac{1}{m^{(\text{train})}} \sum_{i=1}^{m^{(\text{train})}} (y_i^{(\text{train})} - \hat{y}_i^{(\text{train})})^2 = \frac{1}{m^{(\text{train})}} \sum_{i=1}^{m^{(\text{train})}} (y_i^{(\text{train})} - \mathbf{w}^T \mathbf{x}_i)^2$$

To minimize $\text{MSE}_{\text{train}}$, set the gradient to zero:

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0 \Rightarrow \text{Normal equations: } \mathbf{X}^{(\text{train})T} \mathbf{X}^{(\text{train})} \mathbf{w} = \mathbf{X}^{(\text{train})T} \mathbf{y}^{(\text{train})}$$

Explicit solution:

$$\mathbf{w} = (\mathbf{X}^{(\text{train})T} \mathbf{X}^{(\text{train})})^{-1} \mathbf{X}^{(\text{train})T} \mathbf{y}^{(\text{train})}$$

With this procedure we can find the result for our minimization problem, which consists in a specific vector of weights.

But we can find the same solution in an alternative way, by plotting the MSE as function of the weights and then finding its minimum by setting the first derivative equal to zero.

The MSE loss is convex in w

$$\text{MSE}_{\text{train}}(w) = \frac{1}{m} \sum_{i=1}^m (y_i - wx_i)^2$$

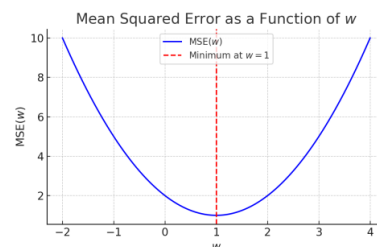
It has the shape of a parabola.

Minimum is found where the derivative is zero:

$$\frac{d}{dw} \text{MSE}_{\text{train}}(w) = 0$$

This is the global minimum since the function is convex.

Note: a function is convex if any line segment between two points on the graph lies above or on the graph itself.



Linear regression with bias:

General case with n features:

where $\hat{y} = \mathbf{w}^\top \mathbf{x} + b = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$, with $\tilde{\mathbf{w}} \in \mathbb{R}^{n+1}$ Bias is added as the last term of the weight vector and it is multiplied by 1, which, in turn, is added as last element in the x-vector. Also in this case, during the training phase we want to minimize the MSE.

$$\tilde{\mathbf{x}} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \\ 1 \end{bmatrix} \in \mathbb{R}^{n+1}, \quad \tilde{\mathbf{w}} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \\ b \end{bmatrix} \in \mathbb{R}^{n+1}$$

After the training phase we have to test our final model, the one obtained by optimizing the weights in order to minimize the MSE on the training dataset, on the test set, so we have to analyze how it behaves in doing predictions: in particular, we have to assess its generalization ability, so how well it performs on new and previously unseen inputs. The generalization error is also known as test error and it must be as low as possible: the lower, the better the model.

$$\text{MSE}_{\text{test}} = \frac{1}{m^{(\text{test})}} \sum_{i=1}^{m^{(\text{test})}} \left(y_i^{(\text{test})} - \hat{y}_i^{(\text{test})} \right)^2$$

In supervised learning, we assume that both the training and test data are generated from a probability distribution over datasets (i.e., input-output pairs (x, y)), called the data-generation process: every time we build a new dataset, we get a different set of instances (but all are sampled from the same underlying distribution $P(x, y)$).

Also, we typically assume that data satisfy the i.i.d. assumptions, so they are independent (each instance in the dataset is generated independently from the others) and identically distributed (training and test instances are drawn from the same probability distribution).

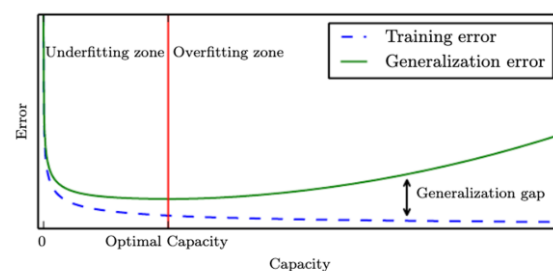
The result of this assumption is that the expected training error of a randomly selected model is equal to its expected test error, but it is important to note that if two instances represent data from the same patient at different time points, this violates the independence assumption due to temporal correlation and, also, if the test set comes from a different population (e.g., different hospital, machine, time period), the data are not identically distributed : this is known as distribution shift, a common real-world challenge.

When using a machine learning algorithm, we do not fix the model parameters in advance and then sample data, instead, we sample a training set, then we use it to choose the model parameters by minimizing the training error and, finally, we evaluate on a separately sampled test set: under this process, the expected test error is greater than or equal to the expected training error.

A good algorithm should make the training error small and keep the gap between training and test errors small. In this sense, the two central challenges in machine learning are underfitting and overfitting: the first corresponds to a poor fit to data with a training error that remains high, while the second situation is characterized by a low training error, but a much higher test error.

Fortunately, we can control whether a model is more likely to overfit or underfit by adjusting its capacity: model capacity is the ability of a model to fit a wide variety of functions, so low-capacity models may struggle to fit the training data, which corresponds to underfitting, while high-capacity model may memorize irrelevant patterns in the training data, which corresponds to overfitting.

We can control model capacity by choosing the hypothesis space of the learning algorithm, which is the set of functions the algorithm is allowed to choose from: changing the number of features and, at the same time, adding new parameters associated to those features is a way to change a model's capacity, since we are changing the hypothesis space, due to the fact that we are adding more terms to our model and they have a higher degree with respect to the one(s) already present, so the set of functions from which the algorithm can sort must consequently change.

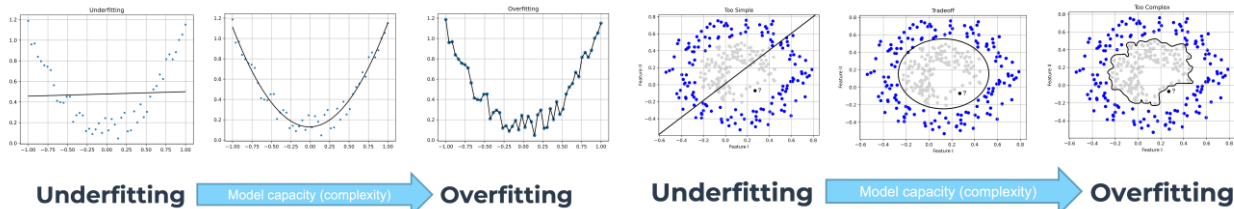


Underfitting

Poor performance on the training data and poor generalization to other data

Overfitting

Good performance on the training data, poor generalization to other data



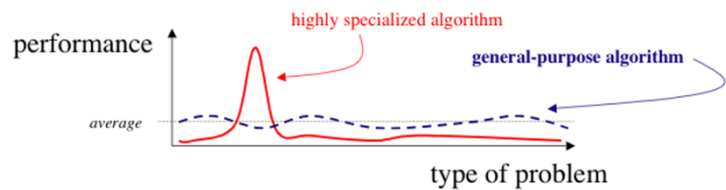
Principle of parsimony - Occam's Razor: among competing models that explain the data equally well, the one with fewest assumptions should be selected, so we prefer simpler models unless complex ones provide significantly better generalization.

In machine learning, this is known as the principle of parsimony: prefer simpler models unless more complex ones provide significantly better generalization.

It is good to apply this principle since simpler models are less likely to overfit, tend to generalize better on unseen data and they are easier to interpret and debug.

Regularization and Hyperparameter optimization:

It is important to remember that no machine learning algorithm is universally any better than any other: the key point is that if one algorithm performs better than another in one class of problems, then it will perform worse in another class of problems.



This is the No Free Lunch Theorem and it implies that we must design ML algorithms to perform well on specific tasks by modifying the hypothesis space, which can be obtained by increasing or decreasing the model's capacity, or by introducing preferences within the hypothesis space, so favoring some solutions over others, unless a preferred solution fits the data significantly better.

An important example of the last strategy is adding weight decay in the training criterion in linear regression, to penalize large weights. In fact, this strategy improves generalization by preventing overly complex models, which can lead to overfitting (the model performs well on training data but poorly on unseen data), by reducing model sensitivity, since, in fact, large weights amplify small variations in the input making the model unstable, and by promoting more robust solutions: small weights ensure that no single feature dominates the prediction and, in this way, the model relies on patterns that are more likely to persist across different datasets, improving robustness and generalization.

	Large weights	Small weights
Model complexity	High (risk of overfitting)	Moderate (better generalization)
Sensitivity to noise	High	Low
Robustness	Poor (unstable predictions)	Good (stable predictions)
Dependence on features	Strong dependence on few features	Balanced reliance on multiple features

The problem of linear regression with weight decay is always a minimization problem, but instead of minimizing just the MSE, we want to minimize the MSE of the training set plus a certain quantity that depends on a parameter lambda, defined a priori, which controls the strength of regularization and is known as a hyperparameter.

$$J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^T \mathbf{w}$$

This function obtained by summing the two quantities described above, since the hyperparameter is fixed in advance, only depends on the weights and, so, its minimization leads to a vector of weights as result, as in the linear regression without the weight decay.

When lambda is equal to 0, we impose no preference for small weights, while increasing its value we force the weights to become smaller. It is important to note that with weights decay we are not changing the degree of the polynomial, since the number of the features in the expression is always the same, but, instead, we are changing their “importance”, their strength, by changing their coefficients (weights).

Expressing preferences for one function over another is a more general way of controlling model capacity than simply including or excluding functions from the hypothesis space, and there are many ways to express these preferences, collectively known as regularization.

Regularization is any modification to a learning algorithm intended to reduce generalization error but not its training error.

Most machine learning algorithms have hyperparameters, so settings that control the algorithm's behavior, which are not learned from the training data: they must be set manually or selected via a validation procedure, like, for example, the degree of the polynomial in linear regression or the strength of the weight decay.

In weight-decayed linear regression, we minimize:

$$J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^T \mathbf{w}$$

Interpretation:

- $J(\mathbf{w})$ expresses a preference for solutions with smaller squared L_2 norm of the weights.

Examples:

- Single feature:

$$\hat{y} = w_1 x_1$$

$$J(w) = \text{MSE}_{\text{train}} + \lambda w_1^2$$

- Two features:

$$\hat{y} = w_1 x_1 + w_2 x_2$$

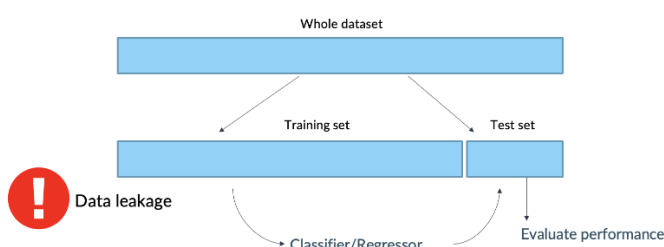
$$J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda (w_1^2 + w_2^2)$$

Note:

- λ is a hyperparameter chosen in advance that controls the strength of regularization.

It is a good idea to optimize hyperparameters, but it is not appropriate to do it using the training set: in fact, if optimized on the training set, hyperparameters would always choose maximum capacity, leading to overfitting.; instead, it is better to use a validation set, so a separate set of examples not seen by the training algorithm.

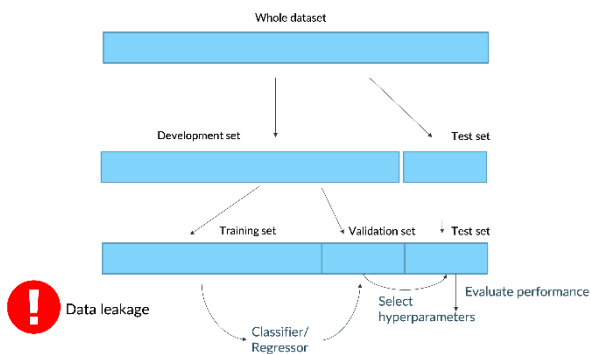
Hold-out validation scheme:



The data set is randomly split into a training set and a data set, both composed of examples drawn from the same distribution and that have never been used to make choices about the model, including selecting hyperparameters.

The model, with hyperparameters fixed in advance, is trained using the training set and the MSE of this training phase is evaluated, then the trained model is used on the test set to make predictions and to compute the generalization error, to assess performance. Simple hold-out validation is not sufficient for hyperparameter optimization: using the test set to tune hyperparameters leads to optimistic bias, so we must introduce a new scheme.

Nested hold-out validation scheme:



The dataset is randomly split into two disjoint parts, the development set and the test set: this is the outer loop. Then, the development set is further randomly split into a training set and a validation set: this is the inner loop.

The training of the algorithm starts in the inner loop, where we want to reach hyperparameter tuning.

In particular, the model is trained on the training set of the inner loop using a specific value for the hyperparameter we want to optimize and its correspondent training error is computed, then the hyperparameter value is changed and the model trained again on the same training set and the correspondent error is evaluated again: this is done for all the possible hyperparameter's values or for all the possible combinations of hyperparameters and, then, the value (or the combination) that makes the training error the lowest is chosen as best hyperparameter.

Once finished this first phase in the inner loop, the model, with its hyperparameter(s) fixed at the value(s) found during the tuning phase, is trained one last time on the development set in the outer loop and then, tested on the test set to make predictions and to estimate its generalization error. With this scheme we have two main problems:

- The number of possible combinations of hyperparameters to test can be very high, so it is required to have a high computational cost.
- A very high amount of data is needed in every set: too small data sets lead to overfitting when having lots of combinations of features and/or parameters.

Also, to ensure independence between test sets and all the other sets, it is necessary to apply some constraints in the splitting phase: if a patient has more features, they must all go into the same set, in this way we avoid data leakage.

Both the Hold-out and the Nested hold-out schemes can be performed in a stratified way, known as Stratified hold-out and Stratified nested hold-out: instead of splitting the data randomly, we split while preserving the class distribution across the training and test sets (and also in the inner loop, so both for hyperparameter tuning and final evaluation).

This is important, especially with imbalanced datasets and ensures that both training and test sets are representative of the original dataset's proportions. The key idea of the nested hold-out scheme is that the inner split is used for making choices, while the outer split is used for an unbiased evaluation of the final decision.

Following these main functions of the two loops, we can easily understand that the nested hold-out scheme can be used for different optimization problems in addition to the hyperparameter optimization explained so far:

- Model comparison: the inner loop is used to select the best model among several candidates, and the outer loop is used to evaluate the generalization performance of the selected model.
- Feature selection: the inner loop is used to select the best subset of features, and then the outer loop estimates the generalization error after this feature selection.

Since the validation set is used to “train” the hyperparameters, its error tends to underestimate the true generalization error, although typically by less than the training error would: it comes that the validation set has some bias and the generalization error, after hyperparameter tuning, should be estimated only on the test set.

Repeated reuse of the same test set over many years can lead to optimistic bias, which means that test results no longer reflect true field performance: the community responds by moving to new and larger benchmark datasets and, as they become saturated, new datasets are adopted to better evaluate true generalization.

Cross-Validation and Nested Cross-Validation:

With biomedical data we have a problem with dataset's size, which, in fact, are too small to apply correctly the nested hold-out or the hold-out schemes.

For this reason, hold-out and nested hold-out are strongly limited when used in the biomedical field: this is why we need other methods for dealing with this type of data, because we cannot have both large test sets and large development sets. The new methods that have been introduced to solve this problem are k-fold cross-validation and k-fold nested cross-validation.

K-Fold Cross-Validation:



It is a technique to estimate a model's generalization error: the dataset is split into k equal-size folds and, for each of them, one-fold is used as the test set, while the remaining $k-1$ folds are used as the training set. This process is repeated k times, with each fold used exactly once as test, and for each of these k cycles the

generalization error is computed, then the final model performance is estimated by averaging the validation results over all the k folds.

The main problem with k -fold cross-validation is the high computational cost, since the training cycle used in the hold-out scheme is repeated k times. Also, it is important to note that, for each cycle, the test set is independent from the training set, this means no data leakage.

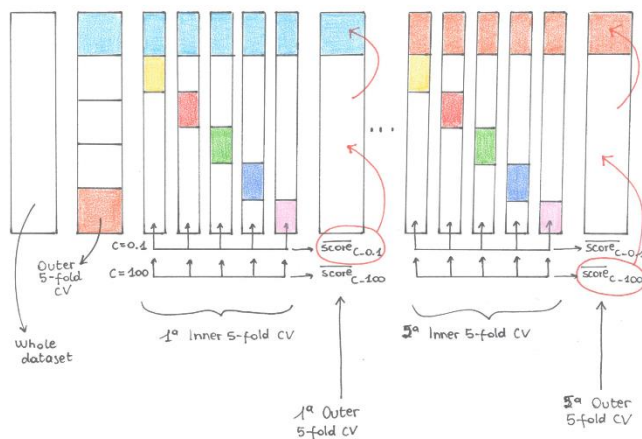
The advantages of this method rely on a more reliable estimate of generalization error, compared to a single validation set, an efficient use of limited data, since every sample is used both for training and for validation, and on the reduction of variance in the performance estimate, compared to a single hold-out split. K is not a hyperparameter that needs to be optimized, statistical studies suggest setting $k = 5$ or, if we have computational time available, $k = 10$, for a good balance between bias and variance.

A special case of the K -fold cross-validation is the Leave-One-Out Cross-Validation (LOOCV): in this case, in fact, k is exactly equal to the number of examples and, so, each fold contains exactly one example for validation. This particular case provides an almost unbiased estimate of the generalization error, but with high variance and high computational cost.

Standard K -Fold Cross-Validation can lead to optimistic bias when hyperparameters are tuned using the same data used for evaluation: nested cross-validation provides an unbiased estimate of generalization error when hyperparameter tuning is involved.

Nested K-Fold Cross-Validation:

The key idea is to separate the process of hyperparameter selection and model evaluation by using two cross-validation loops: an inner loop for hyperparameter tuning and an outer loop for evaluating generalization error.



First, we have to split the data into k outer folds and, for each of them, one-fold will be held out as the test set for evaluation, while the remaining $k-1$ folds will be used to perform hyperparameter tuning.

In fact, in the inner loop, the $k-1$ remaining folds are, in turn, divided into k -folds and a regular cross-validation is performed in order to assess the average error of each hyperparameter's value (or each combination of hyperparameters): the one that is associated

with the lowest error is then chosen to train the model and perform evaluation in the outer loop, in order to find the mean generalization error of the model.

This is done for each of the k outer loops and, after that, the performance across all the outer loops is averaged to estimate the generalization error.

To obtain the final model we perform a new cross-validation over the entire dataset to select the best hyperparameters and, once found, we use them to retrain the model on the entire dataset, without any separated test set, since we do not need it, due to the fact that we already know how much is the generalization error: the model's expected performance is the one estimated by the nested CV. The key point is that the final model is trained on all available data, maximizing its potential while keeping the estimated generalization error unbiased.

Nested CV provides an unbiased estimate of generalization performance, even when hyperparameters are tuned, reduces the risk of optimistic bias caused by overfitting to the validation set, and ensures that the test set remains completely untouched during model selection and training. On the other hand, the tradeoff is the higher computational cost, since multiple models must be trained for each outer loop.

Cross-Validation:

Train → Evaluate on Test Set

Nested Cross-Validation:

Outer Loop: Train on Training Set → Inner Loop: Tune Hyperparameters

→ Evaluate on Outer Test Set

Key difference:

- In CV, no separation between hyperparameter tuning and evaluation: using the same data for both steps may cause optimistic bias.
- In nested CV, hyperparameter tuning and evaluation are strictly separated, leading to an unbiased estimate of generalization error.

MACHINE LEARNING FOR BIOENGINEERING - Mod 2

Clustering:

Its aim, given a set of samples, is to find a partition into a finite number of clusters, so that samples within each cluster are more similar to each other than to those in different clusters: clustering is performed in order to maximize similarities within the same cluster and dissimilarities between different clusters. Some biomedical examples:

- Grouping patients with similar physiological or clinical characteristics.
- Identifying subgroups of patients with the same disease.
- Partitioning medical images into regions with similar characteristics.
- Extracting patterns from large electronic health records.
- Grouping proteins based on structural or sequence similarity.

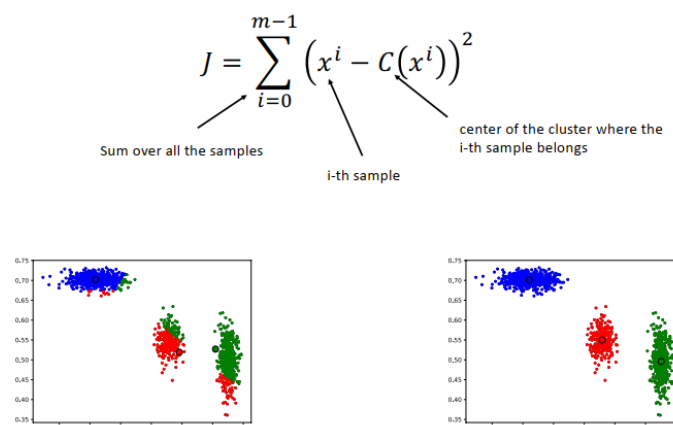
But how do we measure similarity among samples and then, how do we find groups of similar samples? There exist different clustering principles thanks to which we can assess which samples are more like others and, based on that, divide them into subsets.

Compactness-based Clustering:

A cluster is a subset of samples that are close to each other according to a metric. Common similarity metrics are the Euclidean distance, L2 norm, the Manhattan distance, L1 norm, and the Cosine similarity.

In particular, the Manhattan distance is useful when we want to compare samples based on how many different features they have, since it performs well when features represent independent additive quantities, while the cosine similarity is used in high dimensional space, since the relative Euclidean distance becomes less informative.

One example of this type of clustering is the K-Means Algorithm. It starts by defining a priori the number of clusters (K) in which we suppose it is better to divide the dataset, then it generates their centers, for example by selecting K random points and, consequently, each sample is assigned to the closest cluster center, based on the Euclidean distance L2.



After that, it evaluates again the cluster centers and it re-assigns each sample to the cluster whose center is the nearest: in this way, the new centers can be evaluated again and each sample reassigned.

This is done until convergence, according to a predefined stopping criterion: in this sense, the initialization of the algorithm is important, since it converges to a local minimum.

In addition, we must consider that features with large magnitudes dominate the distance computation, so it is necessary to normalize the dataset using standardization to zero mean and unit variance.

The K-Means algorithm partitions data into K clusters by minimizing within-cluster variance, and it works well in the presence of spherical clusters, clusters of similar sizes and clusters with similar densities, while the main limitations are related to the sensitivity to outliers, the detections of nonconvex clusters and the need to define in advance K.

Kmeans++ is the K-Means algorithm but based on an initialization process that improves convergence: it starts by choosing the first center randomly, then it computes the distance from each other point to its closest center and it chooses a new center with a probability proportional to the minimum distance from the other centers.

Model-based Clustering:

A cluster is a subset of samples generated by an underlying probability distribution.

An example of this type of clustering is the Gaussian Mixture Model. It starts with the hypothesis that samples are generated from a mixture of K Gaussian probability distributions, each with probability Π_k , with mean μ_k and covariance matrix Σ_k .

It is similar to K-Means algorithm, since it starts by defining in advance the number of clusters in which we'll divide the dataset, since it is necessary to define the number of Gaussian distributions that we assume have generated our samples, then, for each Gaussian, its metrics, so weight, mean and covariance, are initialized randomly.

After that, the Gaussian Mixture Model applies the Expectation Maximization algorithm that, for each sample, it computes its probability to belong to each of the K different Gaussian distributions pre-defined and, so, it assigns the sample to the cluster to which it has the highest probability to belong.

$$p(x) = \sum_{k=0}^{K-1} \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$$

$\swarrow$ weight of the component (cluster) k $\swarrow$ Normal distribution $\swarrow$ covariance of component k $\swarrow$ centre of component k

$$\mathcal{N}(x|\mu_k, \Sigma_k) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma_k)}} \exp\left(-\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)\right)$$

$$(B) P(k|x) = \frac{\pi_k \mathcal{N}(x|\mu_k, \Sigma_k)}{\sum_{j=0}^{K-1} \pi_j \mathcal{N}(x|\mu_j, \Sigma_j)}$$

probability that sample x belongs to cluster k

$$\begin{array}{l} \text{number of} \\ \text{samples in} \\ \text{cluster k} \end{array} N_k = \sum_{i=0}^{m-1} P(k|x_i) \quad \mu_k = \frac{1}{N_k} \sum_{i=0}^{m-1} P(k|x_i) x_i \quad \Sigma_k = \frac{1}{N_k} \sum_{i=0}^{m-1} P(k|x_i) (x_i - \mu_k)(x_i - \mu_k)^T \quad \pi_k = \frac{N_k}{m}$$

Done this for the samples, all the parameters that completely define a Gaussian distribution are evaluated again for each of the K Gaussians and then, all the samples are re-assigned, and the new metrics are computed another time; this is done until convergence, as for the K-Means algorithm.

It is important to note that, in this way, the K Gaussians will have different weights, according to how many data compose each cluster. Gaussian Mixture Model can deal also with non-spherical clusters and with different-sized clusters too.

Density-based Clustering:

Clusters are identified as dense regions of data space separated by low-density regions.

Density-based Spatial Clustering with Noise:

It searches for regions of high density, and it requires two input parameters: epsilon, which is a distance used to define which samples are considered neighbors, and M, which is a number of points used to define the core points.

Epsilon and M directly define the number of clusters: in fact, the higher the Epsilon the lower the “resolution” with which we are looking at our data and, so, we will obtain few clusters of large dimension; instead, the higher the M parameter, the higher the density with which we want to create our clusters is.

The algorithm starts by finding for each sample the points that are closer than Epsilon (in this way, for each sample we define its density, which will correspond to the number of its neighbors samples), the nearest neighbors, and, then, it defines as a core point each point with more than M nearest neighbors; finally, a random core point, still not assigned to a cluster, is picked and assigned to a cluster, then all the points in the neighborhood are assigned to that same cluster as well: if a core point is added to the cluster, the algorithm goes back and repeat the previous step on that point.

Points not assigned to clusters are labelled as noise.

In another density base algorithm, the Density Peaks Algorithm, what we obtain to separate the samples into clusters are two vectors: one containing the densities associated to each sample, defined as the number of samples that are closer to the point considered than a given threshold epsilon, and the other containing the distances of each point from the closest point that has a higher density than the one considered.

These two vectors can then be plotted: the density vector on the x axis and the distances vector in the y axis. The plot obtained in this way will show separate peaks of density: first, there is an isolated point in the top right position of the plot, then we'll see other points concentrated in a certain part of the plot, under that isolated point, then we'll observe a “jump” and a new isolated point in the top right position, then other concentrated points under it and then, another jump and so go on.

Self-Organizing Maps:

It is a type of artificial neural network that learns to represent input data in a lower-dimensional (usually 2D) discrete lattice and preserves the topological relationships of the input space. It can be described as a set of neurons arranged on a grid (2D), where each neuron has an associated weight vector of the same dimension as the input data: when an input vector is presented, the algorithm finds the neuron whose weights vector is closest to the input (usually Euclidean distance).

This neuron is called the Best Matching Unit (BMU). The BMU and its neighboring neurons are updated, using the following equation: repeating these operations, the neurons specialize to represent regions of space, with nearby neurons representing similar patterns.

$$w_i(t+1) = w_i(t) + \alpha(t)h_{BMU}(i)(x - w_i(t))$$

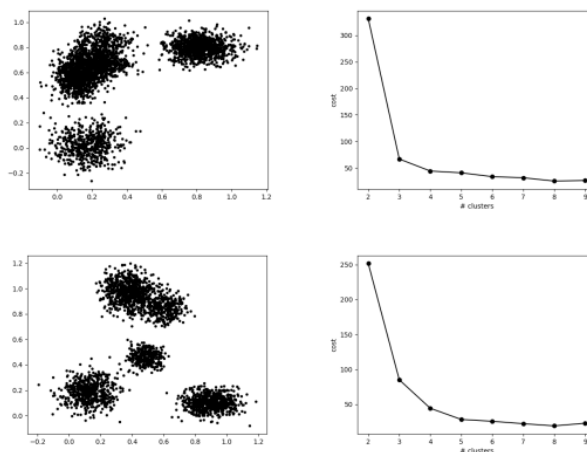
learning rate (decrease over time) decreasing function of the distance from the BMU (gaussian)

In this way, neurons move on the grid towards the samples that have been clustered and presented to the algorithm to train it. Once trained, the network can be used to cluster new data: the number of clusters could be, for example, equal to the number of neurons but, usually, it is more common to use the weights of the neurons to perform the cluster, giving them as inputs to a clustering algorithms (as K-means) and then, assigning the sample to the cluster of its BMU.

But how do we define the number of groups? Also in this case, there exist different methods through which we can answer this question:

Elbow method:

It is graphical method in which we graph a cost function that depends on the number of clusters used to perform the chosen clustering algorithm, then we choose the best K value based on a compromise between computational complexity and reduction of the cost: in fact, the higher the K value, the lower the cost function, but also the higher the complexity and the computational cost of the clustering algorithm.

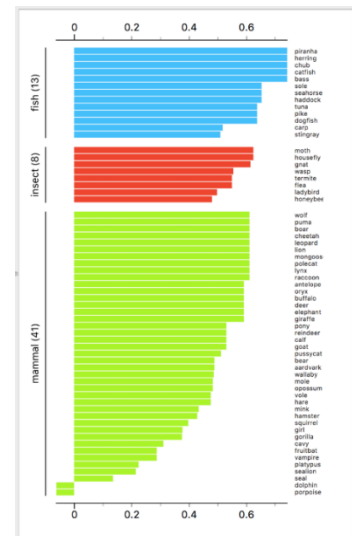


Silhouette Score:

For each point in the dataset is computed (a) the average distance to the points in the same cluster and (b) the average distance from the closest cluster, then the silhouette score is evaluated as:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

This score ranges from -1 to +1: close to +1 means well-clustered, 0 means overlapping clusters and -1 means likely misclassified. We can compute the average silhouette score for different values of K and choose the one that maximizes it. Silhouette score evaluates both cluster compactness and separation, while Elbow focuses on compactness but, also, it has a lower computational cost.



Akaike Information Criterion (AIC):

It is used to compare and select models by balancing goodness-of-fit against model complexity; it is not an absolute score; it can be used only to compare models fitted using the same dataset.

$$AIC = 2k - 2\ln(L)$$

number of parameters of the model: this terms penalizes complex models with a high number of parameters

maximum likelihood of the model: this term rewards model that better fits the data

Bayesian Information Criterion:

It is a criterion for model selection that balances model fit and model complexity. It is similar in purpose to AIC but imposes a stronger penalty for additional parameters, especially as the sample size increases.

number of samples

$$BIC = k \ln(m) - 2 \ln(L)$$

number of parameters of the model: this term penalizes complex models with a high number of parameters

maximum likelihood of the model: this term rewards model that better fits the data

When using AIC or BIC, we choose as K the value that minimize these parameters.

We can then evaluate the goodness of our clustering techniques by using the Adjusted Rand Index, which measures how similar two clustering schemes are, correcting for agreement that could occur by chance: the game changer is to compare the result obtained with a ground truth.

For each pair of samples there are four possibilities:

1. The two samples are in the same cluster both in y_{true} and y_{pred} (a)
2. The two samples are in different clusters both in y_{true} and y_{pred} (b)
3. The two samples are in the same cluster in y_{true} and in different clusters in y_{pred} (c)
4. The two samples are in the same cluster in y_{pred} and in different clusters in y_{true} (d)

The Rand Index, RI, is defined as: $(a + b)/(a + b + c + d)$.

The ARI is a value between 0 and 1: zero means random agreement or disagreement, while one means that the two compared clustering are identical.

$$ARI = \frac{RI - \langle RI \rangle}{\max(RI) - \langle RI \rangle}$$

↖ expected RI for random clustering
↘ value of RI when y_{true} and y_{pred} are the same

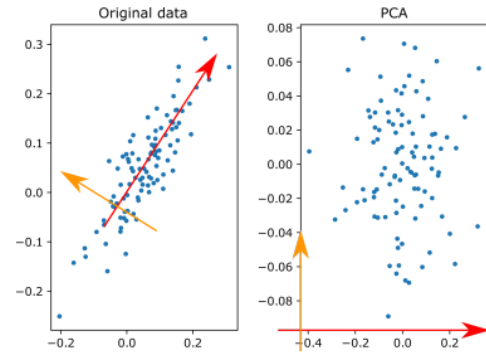
- $ARI = 1 \rightarrow y_{true}$ and y_{pred} are identical
- $ARI = 0 \rightarrow$ random (dis)agreement

Dimensionality Reduction:

In many experiments the number of variables measured is overwhelming, but not all of them are relevant: so, it is important to project the data to a lower dimensional space, in order to lose as little consistent information as possible but, also, to get rid of non-relevant information.

Dimensionality reduction is important also because many algorithms are inefficient when the number of dimensions is high and because having less features that describe the data makes it easier to visualize them.

One example of dimensionality reduction is PCA, through which our aim is to find the orthogonal directions that maximize the variance of the data. We can do this by evaluating the covariance matrix of our original data and then, we find its eigenvalues and eigenvectors: the first eigenvalue is the direction of maximum variance, the second is the second direction of maximum variance, orthogonal to the first one, and so on.

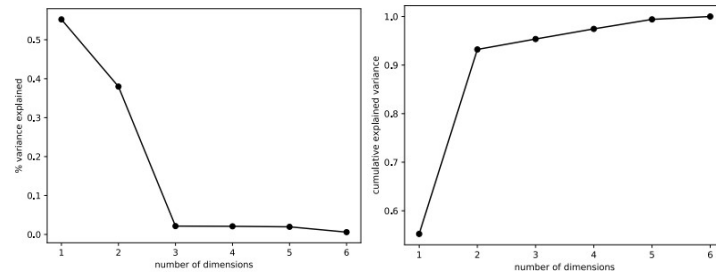


We can choose the first “d” eigenvalues and associated eigenvectors, where d is the number of dimensions in which we want to project our data: they will be our principal components.

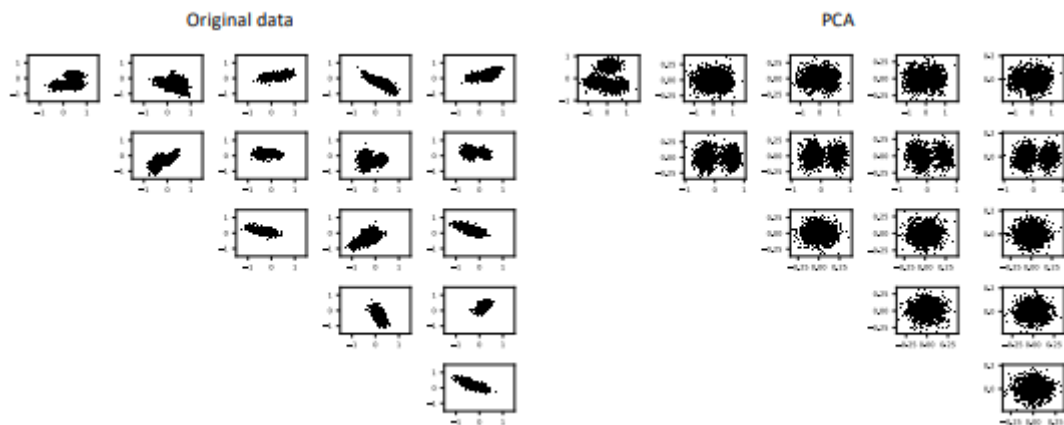
$$\begin{aligned}
 & \text{number of features} \quad \text{1st feature} \quad \text{average of the 1st feature} \\
 & \text{covariance matrix} \quad \text{n x n} \quad \text{covariance matrix} \\
 & COV = \begin{pmatrix} E[(x_0 - \mu_0)(x_0 - \mu_0)] & \dots & E[(x_0 - \mu_0)(x_{n-1} - \mu_{n-1})] \\ \vdots & \ddots & \vdots \\ E[(x_{n-1} - \mu_{n-1})(x_0 - \mu_0)] & \dots & E[(x_{n-1} - \mu_{n-1})(x_{n-1} - \mu_{n-1})] \end{pmatrix} \\
 & \text{correlation matrix} \quad CRR = \begin{pmatrix} \frac{COV_{0,0}}{\sqrt{COV_{0,0}COV_{0,0}}} & \dots & \frac{COV_{0,n-1}}{\sqrt{COV_{0,0}COV_{n-1,n-1}}} \\ \vdots & \ddots & \vdots \\ \frac{COV_{n-1,0}}{\sqrt{COV_{n-1,n-1}COV_{0,0}}} & \dots & \frac{COV_{n-1,n-1}}{\sqrt{COV_{n-1,n-1}COV_{n-1,n-1}}} \end{pmatrix} \\
 & X = \begin{pmatrix} x_0^0 & \dots & x_0^{m-1} \\ \vdots & \ddots & \vdots \\ x_{n-1}^0 & \dots & x_{n-1}^{m-1} \end{pmatrix} \quad \begin{matrix} \text{number of samples} \\ \mu_i = 0 \\ \text{average value of dimension i} \end{matrix} \\
 & \text{1st sample (x^0)} \quad \text{number of dimensions} \\
 & COV = \frac{1}{m} XX^T \\
 & X' = R^T X \\
 & \text{Samples projected in the space of the eigenvectors} \\
 & \text{eigenvalues (in descending order)} \quad \lambda_1 \quad \dots \quad \lambda_n \\
 & R = \begin{pmatrix} | & \dots & | \\ e_1 & \dots & e_n \\ | & \dots & | \end{pmatrix} \\
 & \text{eigenvectors of the covariance matrix}
 \end{aligned}$$

$$\begin{aligned}
 COV' &= \frac{1}{m} X' X'^T = \frac{1}{m} R^T X (R^T X)^T = \frac{1}{m} R^T X X^T R = \frac{1}{m} R^T (m COV) R \\
 &= R^T \begin{pmatrix} | & \dots & | \\ \lambda_1 e_1 & \dots & \lambda_n e_n \\ | & \dots & | \end{pmatrix} = \begin{pmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \ddots & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \dots & 0 & \lambda_n \end{pmatrix}
 \end{aligned}$$

We can plot on the x axis the indexes of the eigenvalues and on the y axis their correspondent values, which represent, as said before, the variance that data have along the direction associated: in this way, we obtain a graph in which we have the percentage of the variance explained according to the number of dimensions used.



Also, it is usually more common to plot the number of dimensions in the x axis and the cumulative explained variance on the y axis. PCA is efficient, easy to interpret and allows you to project new data on previously defined PC, but it is a linear method, so it cannot reduce dimensions if the relations among data are not linear.



$$X = \begin{bmatrix} x_0^0 & x_1^0 & x_2^0 & \dots & x_{n-1}^0 \\ x_0^1 & x_1^1 & x_2^1 & \dots & x_{n-1}^1 \\ \vdots & & & & \\ x_0^{m-1} & \dots & \dots & \dots & x_{n-1}^{m-1} \end{bmatrix}$$

each row is a sample described by n features

$$\text{COV} = \frac{1}{m} X^T X$$

- Our hypothesis is that $\mu_i = 0$
- Then, we can consider the matrix R , composed by the N distinct eigenvectors of the covariance matrix
- Then, we project the data X over the eigenvectors of the covariance matrix: $X' = XR$

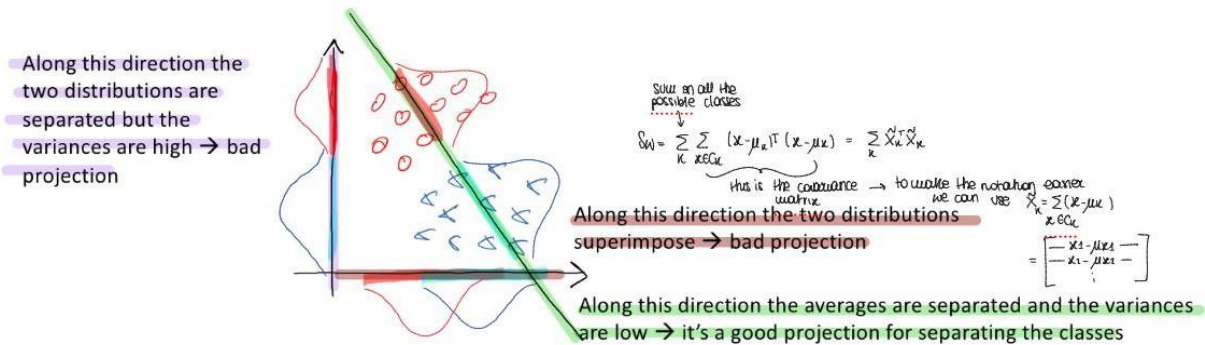
$$\text{COV}' = \frac{1}{m} X'^T X' = \frac{1}{m} (XR)^T (XR) = R^T \text{COV} R = \begin{bmatrix} \lambda_1 & 0 & 0 & \dots & 0 \\ 0 & \lambda_2 & 0 & \dots & 0 \\ \vdots & & & & \\ 0 & \dots & 0 & \lambda_n \end{bmatrix}$$

↑ definition of cov

In this way we are able to find a matrix containing the eigenvalues of the COV matrix

Another strategy is Linear Discriminant Analysis: it is a supervised dimensionality reduction technique that projects data onto a lower-dimensional space while maximizing the distance between class means and minimizing the variance within each class.

Unlike PCA, which focuses on maximizing variance, LDA focuses on maximizing class separability.



- S_W is the covariance within a class: we want it to be low, since it measures how spread out are the classes, instead, we want them to be as confined as possible.

$$S_W = \sum_{k=0}^K \sum_{x \in C_k} (x - \mu_k)(x - \mu_k)^T = \sum_{k=0}^K (X^k - \mu_k)(X^k - \mu_k)^T$$

- S_B is the covariance among classes, which, instead, we want to be high, since it represents how much classes are separated.

$$S_B = \sum_{k=0}^K m_k (\mu_k - \mu)(\mu_k - \mu)^T$$

- X is a matrix with size *number_of_dimensions* $\times$ *number_of_samples*, W is a matrix *number_of_dimensions* $\times$ *number_of_projections* used to change coordinate system.

$$Z = W^T X$$

Now we can find the new covariance within class in the new coordinate system and, also, the new covariance among classes:

$$S'_W = \sum_{k=0}^K (W^T X^k - W^T \mu_k)(W^T X^k - W^T \mu_k)^T = W^T \sum_{k=0}^K (X^k - \mu_k)(X^k - \mu_k)^T W = W^T S_W W$$

Covariance within class in the new coordinate system. The same principle can be used to write the covariance among classes (S_B) in the new coordinate system

$$S'_B = W^T S_B W$$

Finally, we can convert our dimensionality reduction problem into an optimization problem, thanks to the definition of a function $J(W)$ that we want to maximize: its numerator is high if the classes are well separated, while the denominator is low if the classes are compact.

$$J(W) = \frac{|W^T S_B W|}{|W^T S_W W|}$$

So, to maximize it, the main idea is to find its first derivative and then set it equal to zero.

If we multiply W by a scalar constant, both the numerator and the denominator change in the same way: we can define W up to a normalization factor, in order to impose the denominator to be equal to 1 and maximize the numerator. Using the Lagrange Multipliers we can define an alternative function:

$$L(W, \lambda) = |W^T S_B W| - \lambda(|W^T S_W W| - 1)$$

Considering just a 1D projection: $L(W, \lambda) = W^T S_B W - \lambda(W^T S_W W - 1)$

As said before, we can take its first derivative and set it equal to zero:

$$J(W) = \frac{W^T S_B W}{W^T S_W W} = \frac{\lambda W^T S_W W}{W^T S_W W} = \lambda$$

$$S_B W = \lambda S_W W$$

From this formula we can see that the highest eigenvalue is the one that maximizes the function $J(W)$: in case we want to project along different directions we can apply the same argument to the following eigenvectors.

The matrix S_b is a linear combination of rank-1 matrixes, since the product of one vector by its transpose is rank-1, so its maximum rank is $k - 1$: this means that it is possible to project at maximum over a space with dimensions equal to the number of classes minus 1.

Multi-Dimensional Scaling (MDS): the main idea is that this dimensionality reduction algorithm is like having a list of cities that we don't know where they are but, at the same time, we know their distances from each other. We want to put them on a map, preserving the known distances.

So, MDS is a dimensionality reduction technique that embeds data points into a lower dimensional space while preserving the pairwise distances between points as closely as possible. It needs as input the matrix of pairwise distance, so it can be used even when the actual coordinates of the samples are unknown; in fact, each element of the input matrix is: $d_{ij}^2 = |x^i - x^j|^2 = |x^i|^2 + |x^j|^2 - 2 x^i \cdot x^j$

We can transform D^2 into a matrix of inner products by removing the average from columns and rows:

$$B = -\frac{1}{2} H(D^2)H$$

$$H = I - \frac{1}{m} \mathbf{1}\mathbf{1}^T$$

Once obtained matrix B , we can evaluate its eigen decompositions since, in this way, we can reconstruct the original coordinates starting from the matrix of distances, with arbitrary origin of the reference system.

$$B = X^T X = V \Lambda V^T \quad X = \Lambda^{1/2} V^T$$

By selecting only the first p -dominant eigenvector, we project on the reduced space that better preserves the pair-wise distances.

$$X' = \Lambda_p^{1/2} V_p^T$$

IsoMap: for each data point the algorithm connects it to its K nearest neighbors, or within a radius epsilon, forming a graph where the nodes are the data points while the edges are the local distances.

Then, the algorithm computes the shortest paths between all pairs of points on the graph and, finally, the MDS is applied to these distances.

T-Distributed Stochastic Neighbor Embedding (t-SNE): the aim is to project data into a lower dimensional space where samples that are similar in the original space are close to each other in the reduced space, and samples that are dissimilar in the original space are distant in the reduced space.

So, the main intuition is that we want to measure how similar points are to each other and, to do this, the main idea is to define a probability distribution: in particular, we assume that, in order to have a point in a specific plane, there must be an underlying distribution of which that point is the manifestation; this means that there is a certain probability for all the points to be observed in the specific place of the space in which they are.

But how can we find the discrete probability distributions associated to each point in the dataset?

We evaluate for the j-th point its probability to have been generated by the same underlying probability distribution from which the i-th point comes from, which means that we find their probability to be near in the original space.

The probability of j given i is the ratio between the probability that i generates j and the probability that i generated any other sample.

$$P_{j|i} = \frac{e^{-\frac{d_{ij}^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{-\frac{d_{ik}^2}{2\sigma_i^2}}}$$

(σ_i defines the width of the gaussian distribution)

Finally, the similarity between samples i and j is computed as:

$$p_{i,j} = \frac{P_{j|i} + P_{i|j}}{2N}$$

In this way, we can convert our dataset into a set of discrete probability distributions.

So, now, we can project the original dataset into a lower dimensional space, in a random way, and there, associate again to each point their corresponding probability distributions, known as Q.

$$q_{i,j} = \frac{(1 + d_{ij}^2)^{-1}}{\sum_k \sum_{l \neq k} (1 + d_{kl}^2)^{-1}}$$

It is similar to the measure of similarity in the original space but using the t-SNE distribution with 1 degree of freedom instead of the gaussian distribution, since it has a longer tail than the gaussian distribution, and this prevents an excessive crowding of the samples in the low-dimensional space.

Since in the reduced space we should preserve as much as possible the information of the original space, the Q probability distribution should be like the P ones. So, we must compare them using the Kullback-Leibler divergence:

$$D_{KL}(P|Q) = \sum_i P_i \log_2 \frac{P_i}{Q_i}$$

In fact, it measures the difference between the two compared distributions: in particular, if P is the real distribution and Q is the model approximation, D measures the lost information when Q is used to approximate P. If P and Q are very different, if D is high, the points must be moved in the reduced space.

So, to conclude, the t-SNE algorithm takes as input a perplexity value that is used to set the variance value for each point: in particular, the perplexity wanted is fixed and then, for each point the σ_i value that gives that perplexity is found, so that σ_i adapts to the local density of samples.

After that, as said before, the algorithm finds for each point its underlying probability distribution and the dataset is then projected with random positions in the low-dimensional space, where new underlying probability distributions are found.

The Kullback-Leibler divergence is then evaluated between all the possible P_i and Q_i , and the steepest descent method is used to minimize the D_{KL} value obtained: in particular, the points are moved in the direction opposite to the gradient of D_{KL} .

Uniform Manifold Approximation and Projection (UMAP): maps a high dimensional dataset to a low-dimensional representation and the aim is to reveal clustering in the data.

1. Build a graph for each point including the k-neighbors:

To do this, for each x point the algorithm finds the k nearest neighbors $(y_1, y_2, \dots, y_k)$ and defines a graph with links from x to y_i (i from 1 to k).

Then, it defines the weight of the x - y_i link so that: the sum of the links is equal to $\log_2(k)$ for all the graphs, the weight of the link to nearest point (y_1) is equal to 1 and the weight of the links to the other points is proportional to $[distance(x, y_i) - distance(x, y_1)]$.

The intuition is that the link x - y_i measures the probability that x and y_i belongs to the same cluster: by forcing the sum of weights to be constant, we are stretching the space to have uniform distribution of data.

2. Merge the graph of neighbors, to have a global representation of the dataset:

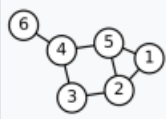
Considering two random samples z_1 and z_2 , only three possibilities exist:

- The pair z_1, z_2 do not appear in any graph: there is no link between z_1 and z_2
- It appears in only one graph: this means that the link between z_1 and z_2 has weight corresponding to the only value available
- It appears in two graphs: in this case the weight is set as $1 - (1 - p_1) * (1 - p_2)$, with p_1 and p_2 corresponding to the weights in the two graphs where the pair z_1 and z_2 appears.

The intuition is that $(1 - p_1)$ is the probability that z_1 and z_2 are not linked in one of the graphs and the same for $(1 - p_2)$, with respect to the other graph. So, considering them as independent events, $(1 - p_1) * (1 - p_2)$ is the probability that in both graphs the two samples are not linked: 1 minus this value estimates the probability that they are linked.

3. Create a representation of the graph in a low dimensional (2D) space

The algorithm defines the Laplacian matrix of the graph and computes its first two non-trivial eigenvectors, then it takes the N components of these eigenvectors as initial coordinates of the samples.

Labelled graph	Degree matrix	Adjacency matrix	Laplacian matrix
	$\begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{pmatrix}$

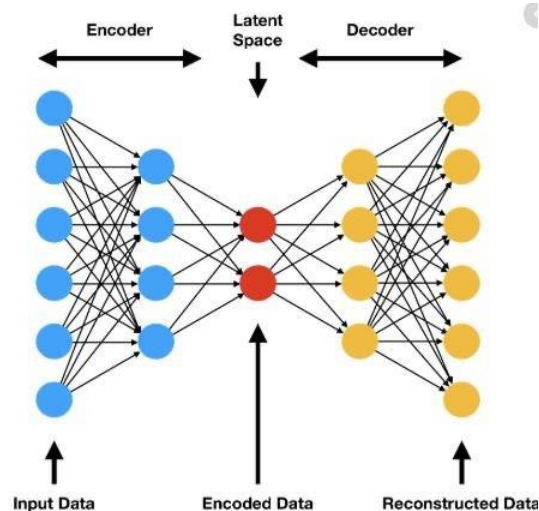
This first representation of the graph is improved by using cross-entropy to estimate the difference between the graph and its low dimensional representation, which, in fact, is minimized.

$$S = - \sum_{e \in E} p_e \log \frac{p_e}{q_e} - (1 - p_e) \log \frac{(1 - p_e)}{(1 - q_e)}$$

set of edges
probability that the edge exist in the original graph
probability that the edge exist in the low dimensional representation

AutoEncoders for Dimensionality Reduction: the ANN is trained to minimize the distance between input data and reconstructed data. The latent space might be poorly regularized. (e.g. samples that are close in the latent spaces might correspond to input data highly different)

Variational AutoEncoders are a possible solution to foster regularization of the latent space.



Regression/Classification with Linear Models:

There exist two main types of regression: the linear regression and the logistic regression

- Linear Regression: $y \in \mathbb{R}$

$$h_{\beta}(x) = \sum_{i=1}^n x_i \beta_i = x^T \beta$$

How can we optimize the model? We define a cost function: $J(\beta) = \sum_{i=1}^m (h_{\beta}(x_i) - y_i)^2$, and we want to minimize it by taking the gradient and setting it equal to 0

$$\frac{\partial J(\beta)}{\partial \beta} = \sum_{i=1}^m 2(h_{\beta}(x_i) - y_i) \frac{\partial h}{\partial \beta} \longrightarrow \text{It can be minimize by: (a) using the conjugate gradient method}$$

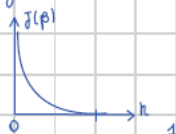
(b) setting it equal to 0 and resolve through Normal Equations

- Logistic Regression: $y \in \{0, 1\}$

$$h_{\beta}(x) = \frac{1}{1 + e^{-x^T \beta}} = \text{probability for the } i\text{-th sample with features } x_i \text{ to be in an output class}$$

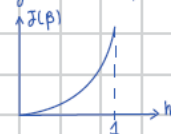
$$J(\beta) = \sum_{i=1}^m -y_i \log(h(x_i)) - (1 - y_i) \log(1 - h(x_i)) \longrightarrow \text{This cost function has sense since, in fact, if we assume:}$$

$$1) y_i = 1 \longrightarrow J(\beta) = -\log(h(x_i))$$



We know that our sample is in class 1, so, every time that $h(\beta)$ is far from this value, which means having a "non-correct" model, we'll pay a cost that is higher the more the result is far from 1

$$2) y_i = 0 \longrightarrow J(\beta) = -\log(1 - h(x_i))$$



We know that our sample is in class 0, so, every time that $h(\beta)$ is far from this value, which means having a "non-correct" model, we'll pay a cost that is higher the more the result is far from 0

In finding our logistic/linear regression model, we want to avoid overfitting, so we can:

- Manually reduce the number of features
- Exploit a regularization method: Keep all the features, but reduce the value of the parameters

REGULARIZATION

$$1) \text{ Ridge - } L_2: \text{ we add the term } \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2 \text{ to our cost function} \longrightarrow J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2 + \frac{\lambda}{2} \sum_{j=1}^n \theta_j^2$$

$$2) \text{ Lasso - } L_1: \text{ we add the term } \frac{\lambda}{2m} \sum_{j=1}^n |\theta_j| \text{ to our cost function} \longrightarrow J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2 + \frac{\lambda}{2} \sum_{j=1}^n |\theta_j|$$

This second type of linearization is used when we want to set the non-relevant features equal to 0

$$3) \text{ Elastic net - } L_1 + L_2: J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2 + \frac{\lambda}{2} \left((1-\alpha) \sum_{j=1}^n \theta_j^2 + \alpha \sum_{j=1}^n |\theta_j| \right)$$

4) Firth's Logistic Regression

Developed to deal with the problems created by using the logistic Regression for:

- small datasets
- imbalanced datasets
- complete separation of classes along one feature: the model, in fact, tends to $\pm \infty$ depending on values of the independent variables, and consequently the parameters diverge

$$\text{The cost function is: } J(\theta) = J'(\theta) - \frac{1}{2} \log |I(\theta)|$$

FISHER INFORMATION MATRIX: it measures how sensitive is the likelihood to changes in parameters

The idea is that if a small change in a parameter greatly change the likelihood, that parameter is highly informative

$$I(\theta)_{ij} = - \mathbb{E} \left[\frac{\partial^2 \log L}{\partial \theta_i \partial \theta_j} \right]$$

Now we have to find the likelihood: we suppose that the output is sampled from a Bernoulli distribution with probability p_i , depending on the input features, according to the logistic function

$$y_i \sim \text{Bernoulli}(p_i) \text{ with } p_i = \frac{1}{1 + e^{-\theta^T x_i}} \longrightarrow \text{the } i\text{-th output has the probability to be observed: } y_i \sim p_i^{y_i} (1 - p_i)^{1 - y_i}$$

$$\text{Considering samples independent: } L(\theta) = \prod_i p_i^{y_i} (1 - p_i)^{1 - y_i}$$

$$\text{Once found the likelihood function, we can linearize it using the Taylor series: } L(\theta + \delta) \approx L(\theta) + \nabla_{\theta} L(\theta) \delta + \frac{1}{2} \delta^T H \delta$$

If the determinant of the information matrix is high, there's high curvature of the log-likelihood in the proximity of the maximum, which corresponds to high sensibility to the values of the parameters

How can we carry out a classification task when the classes are more than two?

- Multi-class classification / One-Vs-All: the idea is to train N models that classify each class versus the other ones and then, choose the class with the highest probability.
- Multi-class classification / One-Vs-One: train $N * (N - 1)/2$ models that classify each class versus each other class, then choose the class that wins most duels.
- Softmax regression/ Multinomial logistic regression: a score is computed for each class, and thanks to it, a probability is estimated for each class using the Softmax function:

$$s_k(x) = \theta_k^T x \quad p_k(x) = \frac{\exp(s_k(x))}{\sum_j \exp(s_j(x))}$$

Each class has its own weights vector

Thanks to these probabilities, it is possible to find cross-entropy as a function of parameters: in this way, it is possible to find the best set of parameters by minimizing the cross-entropy function.

What is Cross-entropy? To define this concept is first necessary to introduce the concept of entropy.

Entropy is the average amount of information that you get from one sample drawn from a given probability distribution, which corresponds to how many bits, on average, are needed to store information. Entropy can be seen also as how much of our ignorance about an event is decreased by knowing the outcome.

$$H(p) = - \sum_i p_i \log_2 p_i$$

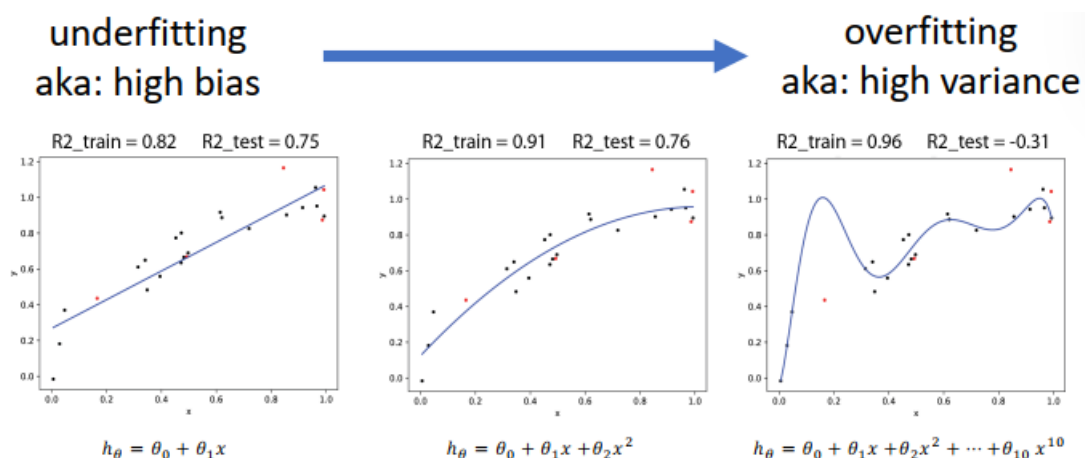
The cross-entropy is defined as:

$$H(p, q) = - \sum_i p_i \log_2 p_i + \sum_i p_i \log_2 \frac{p_i}{q_i} = H(p) + KL(p, q)$$

↑
how many bits I need to encode the outcome using the wrong model q
↑
entropy of the true distribution
↑
KL divergence: mismatch if p is approximated by q

Cross-entropy is composed, for each sample, by a term correspondent to the entropy of the real probability distribution and another term that evaluates how far, the predicted probability distribution, so the one produced by our model, is from the real one: this is the KL divergence.

Minimizing the Cross-entropy means minimizing the KL divergence.



Decision Trees and Ensemble Methods:

A decision tree is a supervised machine learning model used both for classification and regression tasks. The model, starting from a root node representing the entire dataset, splits data step by step, using binary decisions: the groups found for each step are internal nodes, while the groups found at the end of the process, so the terminal nodes, are known as leaf and represent where the final prediction is made.

In case of regression, the final prediction is the average value in the leaf node, while in the case of classification the final prediction is the most common class in the leaf node.

The aim of these splitting is to create terminal nodes that are as homogeneous as possible: in classification, we want to obtain leaves that include samples of the same class, while in regression we want leaves including samples with similar output.

CART (Classification and Regression Tree algorithm): it constructs a tree by recursively splitting the dataset into subsets, based on feature values, with the goal of making each subset as homogenous as possible.

To evaluate the homogeneity of each subset, a “purity” score is computed:

- For classification it corresponds to the Gini impurity score, and it measures how homogeneous the class labels are.

Gini impurity for one leaf node

$$G = \sum_k p_k \sum_{i \neq k} p_i = \sum_k p_k (1 - p_k) = \sum_k p_k - p_k^2 = 1 - \sum_k p_k^2$$

probability of being in another class (in that node)

sum over the classes

probability of being in class k (in that node)

- For regression it usually corresponds to the MSE (or MAE) and it measures how similar the numeric values are.

- Mean Squared Error (contribution for each leaf node)

$$J = \frac{1}{m_k} \sum_i (y_i - \bar{y}_k)^2$$

number of samples in the leaf node

average of the output in the leaf node

sum over the samples inside the leaf node

- Mean Absolute Error

$$J = \frac{1}{m_k} \sum_i |y_i - \bar{y}_k|$$

Gini impurity quantifies the probability that a randomly chosen sample would be incorrectly classified if it were labeled according to the class distribution in that node.

- $G = 0$ means that the node includes only one class, perfect purity.
- $G = 0.5$ means, in the case of two classes, that the node includes the same number of samples for each class.

Algorithm step by step:

1. Define a root node including all the samples in the dataset.
2. For every feature and possible split threshold evaluates how well the split separates the data, using the Gini impurity for classification tasks and the MSE for regression tasks.

Pay attention to the fact that we have to compute an “impurity” score of the splitting, so we have to consider both the two subsets (nodes) obtained through the splitting criterion when evaluating the Gini impurity or the MSE: we can take a weighted average of the scores obtained for each of the two nodes created with the splitting.

$$G_{split} = \frac{m_L}{m} G(L) + \frac{m_R}{m} G(R)$$

number of samples in the left node
 impurity of the left node

3. Select the best splitting, so the one with the lowest “impurity”.
4. Divide the dataset into two subsets using the best splitting criterion previously found.
5. Go back to point 2 until the nodes are pure or a stopping criterion is satisfied

The final complete tree might cause overfitting: with pruning we try to balance fit quality with the model complexity.

Consequently, cost complexity pruning is a regularization method that aims to reduce overfitting.

In particular, with cost complexity pruning a score α_t is computed for all the internal nodes, then the node with the minimal score is found: since the numerator of this score is the change in the training error, provoked by choosing a “reduced” tree with respect to the full tree (in this sense, the training error will be always higher in the pruned tree with respect to the full tree), and the denominator is the change in complexity, we are selecting the node that gives a minimal contribution with respect to its complexity.

$$\alpha_t = \frac{R(T) - R(T_t)}{|T_t| - 1}$$

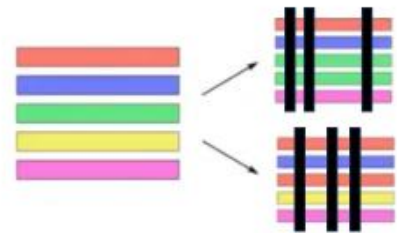
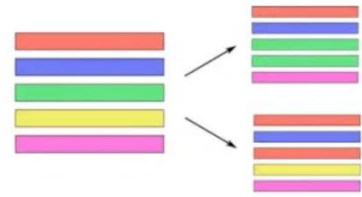
training error of the tree (misclassification error or MSE)
 training error of the full tree
 number of leaf nodes in full tree
 because if we prune the tree the complexity is reduced by 1. The denominator is the reduction in complexity

Once found the node with minimal score using cross-validation, the algorithm prunes that subtree and repeats the operation. In any case, more models are better than one: so, instead of creating a single tree, it is better to create numerous trees and then, combine their predictions.

The classification predictions can be combined using soft-voting, in which the average of the probabilities is used, or hard-voting, in which the most voted class is selected.

There exist different ensemble models that perform this aggregation of decision trees:

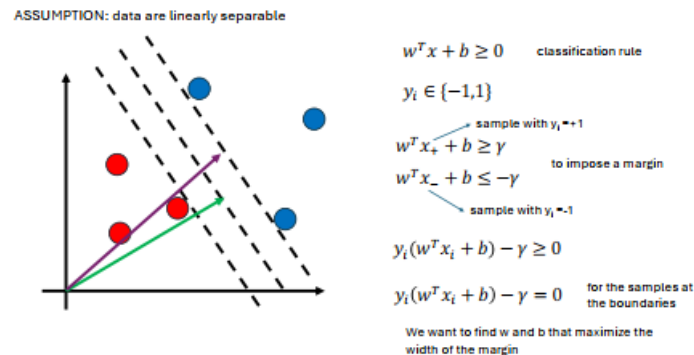
- Bagging (Bootstrap Aggregating): different datasets are generated by Bootstrap with replacement from the same starting dataset, then one decision tree is trained for each different dataset and, finally, the predictions of each trained decision tree are aggregated.
- Random Forest: the core idea is that ensemble of models works better when the models are less correlated, so, a possible strategy is to train models using not only different sets of samples, but also different sets of features. In particular, with random forest, multiple datasets are created by sampling the original training dataset with replacement, then, for each dataset, some features are removed and a decision tree is trained; once done that, predictions are aggregated.
- ADABOOST (Adaptive Boosting): instead of taking the samples randomly, as in bagging, in boosting the model focuses on the samples that are hard to classify. In particular, all the samples are initialized with the same weight, then a weak learner is trained with them and the weight error is computed, which is used to compute the model weight (high for good models), in turn exploited for updating the weights of the samples, which will increase if the samples are misclassified. The process is then repeated selecting samples according to their weights.
- Gradient Boosting: instead of re-weighting data, gradient boosting fits each new model to the residual errors: in particular, a very simple model is initially used to fit the data, then the residuals, so the errors, are computed and a new weak learner is trained to predict that residuals; the model is then updated.



Support Vector Machines:

The aim is to separate data into two classes, starting from the assumption that they are, in fact, linearly separable. The main idea to do this is that the best boundary is the one that gives the highest margin between the two classes.

Given this classification rule, we want to find w (direction orthogonal to the separation direction of the dataset) and b (threshold used for the separation of the dataset) in order to maximize the width of margin, since it means finding the boundary that gives the highest margin.



The width of the margin: $(x_+ - x_-) \cdot \frac{w}{|w|} = \frac{2\gamma}{|w|}$

Gamma is a multiplicative factor, so we can assume it equal to 1 without a loss of generality. So, to make the margin wide (Large Margin Classifiers), we need to minimize the norm of w , which is the same as minimizing $0.5 * |w|^2$, that, in turn, can be done by using the Lagrange multipliers:

$$L = \frac{1}{2} |w|^2 - \sum \alpha_i [y_i(w^T x_i + b) - 1]$$

This sum is over all points, but the constraint is strictly positive for points not in the margin and, so, in this case α needs to be zero, while it needs to be strictly positive for samples at the boundary.

By substituting the w and b values that minimize L both in L and in the classification rule, we can observe that they become dependent only on the scalar products of the samples.

$$\begin{aligned} \frac{\partial L}{\partial w} = w - \sum \alpha_i y_i x_i & \xrightarrow{\text{setting the derivatives to zero}} w = \sum \alpha_i y_i x_i \\ \frac{\partial L}{\partial b} = - \sum \alpha_i y_i & \xrightarrow{\text{setting the derivatives to zero}} \sum \alpha_i y_i = 0 \end{aligned}$$

From this consideration, we understand that we can exploit the Kernel “trick”: we can assume that a mapping exists from the original space to a new dimensional space, usually high dimensional, and then, we can define the function K , which is the Kernel, that gives the scalar product in the high dimensional space.

Since the algorithm depends only on scalar product, if we know K we don't need to work in the high dimensional space.

For a function to be a Kernel, it needs to exist a mapping function:

In order for a function to be a Kernel, it needs to exist a mapping function $x \rightarrow \phi(x)$ such that $K(x, z) = \phi(x)^T \phi(z)$

Mercer's theorem. A function is a Kernel if and only if given any set of points $x_1, \dots, x_d$ the matrix with elements $K_{ij} = K(x_i, x_j)$ is positive semi-definitive

Mercer's theorem tells us that a function is a Kernel if and only if, given any set of points, the matrix with elements $K_{ij} = K(x_i, x_j)$ is positive semi-definite.

An example of a Kernel function is the Gaussian Kernel. (Radial Basis Function Kernel)

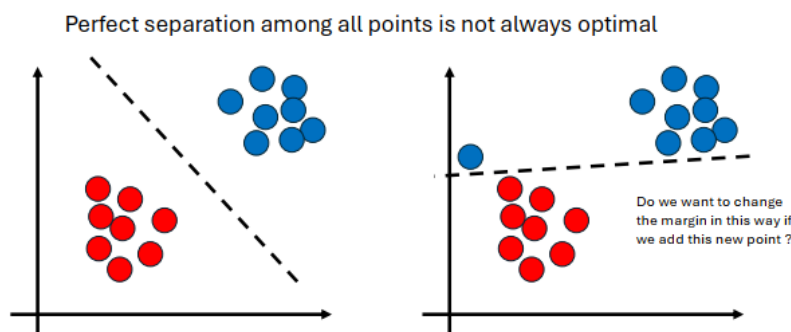
$$K(x, z) = e^{-\frac{\|x-z\|^2}{2\sigma^2}}$$

It measures the similarity between two samples, and it maps to an infinite dimension dataspace.

The Kernel trick can be used in any algorithm that depends on input features only through the scalar products between samples, so for instance in PCA: use the Kernel to compute the covariance matrix, which must be then centered by subtracting means over columns and rows and, then, by adding back the global mean; after that, eigenvalues and eigenvectors of the covariance matrix are computed and, so, data are projected into the principal components.

The problem is that the principal components are not interpretable.

Soft Margin Classifier: the idea is that perfect separation among all points is not always optimal.



We can modify our classification rule in order to admit also some points inside the margin:

$$y_i(w^T x_i + b) - 1 \geq -\xi_i, \xi_i > 0$$

Then, we have to minimize the number of points inside the margin, so, as before, we can exploit the Lagrange multipliers to find an equivalent function easier to derive and set it equal to zero:

$$f = \frac{1}{2} |w|^2 + C \sum \xi_i$$

Low C means having a high regularization, since we give a lower penalty to the points inside the margins, while a high C means low regularization for the opposite reason.

Soft margin classifier is a regularization method that avoids overfitting in the SVM.

Explainability:

Explainable AI (XAI) is a branch of artificial intelligence focused on making AI systems understandable to humans: instead of acting like a “black box” that gives answers without explanation, explainable AI shows how and why a model made a decision.

The aim of these methods is to identify which features are most important for the prediction in linear models, the coefficients provide a direct estimate of feature importance, and in Decision Trees the feature importance can be estimated based on its contribution to reduction in impurity; but in nonlinear models features importance is less directly observable.

Common feature attribution methods are:

- LIME (Local Interpretable Model-Agnostic Explanations): the core idea of LIME is that even complex models behave approximately linearly in a small region around a single prediction. LIME provides a local explanation for a specific prediction, rather than a global understanding of how the model behaves.

Algorithm:

1. Generate perturbed samples close to the original input.
 2. Evaluate the original model on these new samples.
 3. Fit a simple interpretable model locally using the perturbed inputs and their prediction: the “simple interpretable model” could be, for example, a linear model or a decision tree.
 4. Samples closer to the original input receive higher weights.
 5. Use the surrogate model to generate explanation.
- SHAP (Shapley Additive Explanations): the SHAP method explains a prediction by answering “How much did each feature contribute to pushing the model output away from the average prediction?”. It is based on the calculation of Shapley values considering the features as players and prediction as reward.

Its main limitation is linked to the high computational cost and, also, to the global interpretability, since combining SHAP values is complicated.

The diagram illustrates the SHAP formula and its components. On the left, the formula for the Shapley value $\phi_i(x)$ is shown:
$$\phi_i(x) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f_x(S \cup \{i\}) - f_x(S)]$$
 Each part of the formula is annotated with a box:

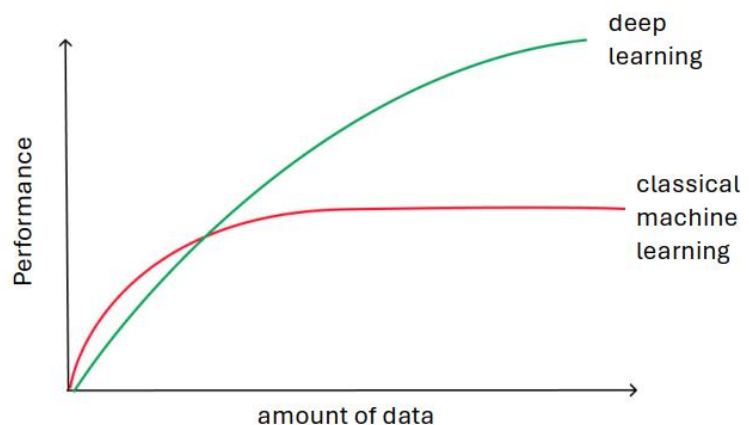
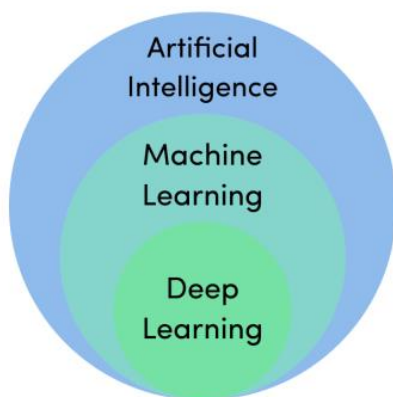
- $|S|$: number of features in S
- $|F|$: total number of features
- $f_x(S \cup \{i\})$: Prediction of the model in x using the feature set S plus the feature i
- $f_x(S)$: Prediction of the model in x using the feature set S
- $S \subseteq F \setminus \{i\}$: All feature subsets that do not include feature i

 A large blue arrow points from the formula to the right, where the final model prediction is shown:
$$f(x) = E[f(x)] + \sum_i \phi_i(x)$$
 This equation is also annotated:

- $f(x)$: prediction of the model
- $E[f(x)]$: average output
- $\phi_i(x)$: contribution of each feature to move away the prediction from the average

The term $f_x(S)$ is obtained by using the model we’ve implemented for making predictions, which is what we want to explain, for predicting an outcome, which will be, in fact, $f_x(S)$, using as input x a vector whose features that don’t belong to the subset S are random values.

- Clustering
 - KMeans
 - GMM
 - Hierarchical Clustering
 - DBSCAN
 - SOM
 - Elbow/Silhouette/AIC/BIC
 - Adjusted Rank Index
 - Why using metrics different than L2
- Dimensionality Reduction
 - PCA/kernel PCA
 - LDA
 - MDS/IsoMap
 - LLE
 - tSNE
 - UMAP
 - AE/VAE
 - KL divergence
- Linear Models
 - Regression
 - Classification
 - Regularization strategies
 - L2, L1, elastic nets, Ridge
 - Fisher Information Matrix
 - Entropy and Cross-entropy
- Decision Trees
 - Measures of impurity
 - CART algorithm
 - Ensemble methods
 - Bagging
 - Random Forest
 - Adaptive Boosting
 - Gradient Boosting
- Support Vector Machines
 - Large margin classifiers
 - Kernels
 - Regularization
- Explainability
 - LIME
 - SHAP



- Neuron: the single unit of computation
- Weights: strength of each connection
- Transfer function: weighted sum of the neuron inputs
- Activation function: function used to compute the output of the neuron
- Architecture: number and kind of neurons and how they are connected
- Input layer, hidden layer, output layer
- Loss/cost function: function used to train the network (estimate the weights)